WebBeans Developer Guide

2026-10-28

Contents

1. Getting started

1.1 Requirements
1.2 Create an application
1.3 Anatomy of an application jar
1.4 Your first TopComponent and action

2. The mental model

3. Threading — the one rule

4. Services and lifecycle

5. Users, sessions, roles, per-user data

6. Internationalisation

7. Window system

8. Nodes and Explorer — showing data

9. Actions, menus, toolbars, shortcuts

10. Dialogs, notifications, progress, options, help

11. Charts

12. 3D scenes

12a. Graph scenes

12b. Maps

13. Writing your own component

14. Testing

15. Running, configuring, packaging

16. Porting from the NetBeans Platform

WebBeans is a rich web application platform in the spirit of the NetBeans Platform: you write the application in Java 8 (ordinary jars on the class path), the platform renders it in the browser. There is no client code to write for ordinary applications — UI, actions, nodes, charts and 3D scenes are Java objects on the server; a generated JavaScript bundle mirrors them in the browser.

This guide covers concepts and workflows. For a per-component reference with examples, see the **Component Guide** (`docs/component-guide.md`); for a step-by-step tutorial that builds up from a label to a complete application, the **User's Guide** (`docs/user-guide.md`). The **Workbench** sample application (`workbench/` in the repository, `design/sample-app.md`) is the worked example used throughout.

1. Getting started

1.1 Requirements

- JDK 8 (the platform compiles with `--release 8`; later JDKs run it too).
- Maven 3.6+ (or the offline `./build.sh`).
- Any modern browser (ES2017; WebGL for 3D). A tab talks to the server over a WebSocket; long polling is the automatic fallback behind a proxy that cannot forward the upgrade.

1.2 Create an application

Start from the Workbench: copy `workbench/` and keep what you need.

```
myapp/
  pom.xml                  packaging pom; parent org.webbeans:webbeans-parent (manages all
versions)
  hello/                   an ordinary jar: src/main/java (+ resources), the annotation processors
on the compiler path
  app/                     the application jar: branding, the launcher and every jar merged into
one (shade plugin)
```

Build with `mvn package` and run `java -jar app/target/myapp.jar --datadir data` (offline: `./build.sh` produces `build/workbench-dist/workbench.jar` the same way, see §15). Open `http://localhost:8080`, log in as `admin` (the password is printed on the first start, or set `WEBBEANS_ADMIN_PASSWORD`; accounts live in `data/system/users.properties`) — or seed your own accounts from an `Installer` as the Workbench does. The smallest complete application is `examples/hello` (a pom and one action class).

1.3 Anatomy of an application jar

The application is the class path: every jar on it contributes what its `META-INF/services` files list — the registrations the annotation processor generated, `@ServiceProvider` services, `Installer`s. There is no manifest, no module descriptor, no dependency declaration and no isolation between jars; split an application into several Maven projects for the usual reasons (ownership, reuse), not because the platform needs it.

```
hello/
  pom.xml                                    the three annotation processors on the compiler path
(copy examples/hello/pom.xml)
  src/main/java/com/acme/hello/
      HelloTopComponent.java
      SayHelloAction.java
      package-info.java                      package-level registrations (help pages, shared
messages)
      Bundle.properties                      (optional, hand-written keys; never touched by the
processor)
      Bundle_de.properties                   translations — of hand-written and @Messages keys
alike
  src/main/resources/com/acme/hello/hello.svg
  src/main/resources/com/acme/hello/help/hello.md       help pages (§10)
```

The compiler needs the annotation processors on its path: `webbeans-processor` plus `webbeans-windows-processor` and `webbeans-actions-processor` (and NetBeans' `org-openide-util-lookup` for `@ServiceProvider`) in `annotationProcessorPaths`, with `-Awebbeans.module=<name>` naming the package of the generated `WbRegistrations` class (one per jar; `examples/hello/pom.xml` and `workbench/workbench-files/pom.xml` are templates).

Rules worth knowing from day one:

- Order on the class path is the only precedence: when two jars register the same path (`Menu/Tools/x`), the later jar wins and a warning is logged; `@Hide` masks entries deliberately.
- Everything is registered with annotations. There is no XML. The annotation processor runs automatically during `mvn compile` and generates `<codeName>.generated.WbRegistrations`; registration errors (a component without `@Description`, missing bundle key, duplicate path, a registered class without a public no-arg constructor, an action class with two public constructors, a help page without a resource) are compile errors.
- `Lookup` and `@ServiceProvider` are NetBeans' (`org.openide.util`). `@Messages` is WebBeans' own (`org.webbeans.core.i18n.Messages`) with the same syntax — its generated `Bundle` methods use the user's locale, which NetBeans' cannot (see §6).

1.4 Your first TopComponent and action

```
package com.acme.hello;

@TopComponent.Registration(mode = "editor", openAtStartup = true)
@TopComponent.Description(preferredID = "HelloTC", iconBase = "com/acme/hello/hello.svg", helpId =
"hello")
@TopComponent.OpenActionRegistration(displayName = "#CTL_OpenHello")
@Messages({"CTL_OpenHello=Hello", "CTL_Hello=Hello", "LBL_Greeting=Hello, {0}!"})
public final class HelloTopComponent extends TopComponent {
    public HelloTopComponent() {
        setDisplayName(Bundle.CTL_Hello());
        String user = UISession.current().getPrincipal().getDisplayName();
        setLayout(new Layout.Border());
        add(new Label(Bundle.LBL_Greeting(user)), Layout.Border.CENTER);
    }
}

@ActionID(category = "Tools", id = "com.acme.hello.SayHelloAction")
@ActionRegistration(displayName = "#CTL_SayHello")
@ActionReferences({ @ActionReference(path = "Menu/Tools", position = 100),
                    @ActionReference(path = "Shortcuts", name = "D-H") })
@Messages("CTL_SayHello=Say Hello")
public final class SayHelloAction implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        NotificationDisplayer.getDefault().notify("Hello", null, "It works.", null,
                NotificationDisplayer.Priority.NORMAL, NotificationDisplayer.Category.INFO);
    }
}
```

Build, restart, reload the browser: the “Hello” tab is open in the editor area, “Tools ▸ Say Hello” and `Ctrl+H` show a toast, “Window ▸ Hello” reopens the tab after you close it, and `F1` in the tab opens the help page `hello` once you register one (§10).

For a page without the window system, build it from a `UIProvider` (any `@ServiceProvider(service = UIProvider.class)` takes precedence over the platform’s main window, which sits at position 10000):

```
@ServiceProvider(service = UIProvider.class)
@Messages({"LBL_Greeting=Hello, {0}!"})
public final class HelloProvider implements UIProvider {
    @Override public void init(UI ui) {
        Panel page = new Panel(new Layout.Border());
        page.add(new Label(Bundle.LBL_Greeting(ui.getSession().getPrincipal().getDisplayName())),
Layout.Border.NORTH);
        ui.setContent(page);
    }
}
```

2. The mental model

```
   Browser tab  ──transport──►  UI  (component tree, WindowManager, layout)   ← one per tab
   Browser tab  ──transport──►  UI                                          ┐
                                                                            ├── UISession (user, locale, lookup,
executor, user files)
   background job ── session.access(…) ─────────────────────────────────────┘
                                                ┌─ Lookup.getDefault()  (services from every jar)
                                                └─ Registry             (actions, menus, modes,
components, …)
```

- **Component tree**: `Component`s have typed properties and children. Setting a property records a delta; after your code returns, the platform sends one batched message to the browser (a WebSocket frame, or a long-poll reply), which patches the DOM. You never think about the wire.
- **Events** come back only for listener types you registered (`button.addClickListener(…)`, `field.addChangeListener(…)`, `addListener(EventType, …)`); they run on the session's executor, so handlers can touch any component of that session without locking.
- **Lookup** is how things find each other: services (`Lookup.getDefault()`), per-user things (`UISession.current().getLookup()`), and the *selection* (`WindowManager.getDefault().getGlobalContext()` = the activated TopComponent's lookup, usually the selected nodes; `Utilities.actionsGlobalContext()` is the same thing).
- **Registry** is where annotations put their registrations: `Actions/<category>`, `Menu/…`, `Toolbars/…`, `Shortcuts/<key>`, `Windows/Modes`, `Windows/Components`, `Windows/Groups`, `Windows/Layouts`, `StatusBar`, `Nodes/Handles`, `<Type>/Actions` (node popups), `Renderers`, `OptionsDialog`, `Help/Pages`. You rarely touch it directly (`Registry.getDefault().instances(path, type)`, `Actions.forPath(path)` or `RegistryLookup.forPath(path)` when you do); what you see is already filtered for the current user's roles, locale and customisations — on the session executor; a background thread sees everything.

3. Threading — the one rule

All component access happens on the session executor. The platform puts you there automatically in event handlers, TopComponent callbacks, node and explorer callbacks, action `actionPerformed` (unless the action is `asynchronous`), `UI.Timer` tasks, session-bus subscribers, dialog continuations and `PageProvider.render`. From anywhere else — a `RequestProcessor` task, an asynchronous action, a servlet, a message queue listener, a `LookupListener` — hop first:

```
ui.access(() -> label.setText("done"));                    // one tab
session.access(() -> model.update(data));                  // session-level model; UIs observe it
```

Never block the session thread waiting on the browser; there is no synchronous dialog:

```
DialogDisplayer.getDefault().notify(new NotifyDescriptor.Confirmation(Bundle.MSG_Discard(),
NotifyDescriptor.YES_NO_OPTION))
    .thenAccept(result -> { if (result == NotifyDescriptor.YES_OPTION) close(); });   //
continuation runs on the executor
```

Long work goes off-thread with NetBeans' `RequestProcessor` (or `@ActionRegistration(asynchronous = true)`) and comes back with `ui.access`. `NotificationDisplayer`, `ProgressHandle` and `StatusDisplayer` may be called from any thread. Off-thread component *mutation* throws `IllegalStateException` immediately (strict threading is on by default; `UISession.setStrictThreading(false)` only for exotic tests). Nodes follow the same rule: mutate them (and call `setKeys`) on the session executor. Timers: `ui.schedule(task, delayMs)` and `ui.scheduleAtFixedRate(task, initial, period)` return a `UI.Timer` you can `cancel()`; they die with the UI. The Workbench's `PartsModel` is the pattern for a server-side simulation: a daemon scheduler mutates the model under its own lock and publishes through `session.access`.

4. Services and lifecycle

```
@ServiceProvider(service = Installer.class)
public final class WorkbenchInstaller extends Installer {
    @Override public void restored() {              // registry and Lookup complete; seed data,
register listeners, start background jobs
        Path users =
Lookup.getDefault().lookup(DataDir.class).getSystemDir().resolve("users.properties");
        if (!Files.exists(users)) PropertiesAuthenticationProvider.putAccount(users, "alice",
"secret".toCharArray(), "Alice", "user", "admin");
    }
    @Override public boolean closing() { return true; }
    @Override public void close() { /* stop jobs */ }
}
```

`Installer`s are found through `META-INF/services/org.webbeans.core.Installer` (the `@ServiceProvider` above writes that file); `restored()` runs in Lookup order once the platform is up, `closing()`/`close()` in reverse order at shutdown.

Services:

```
public interface GreetingService { String greet(String name); }

@ServiceProvider(service = GreetingService.class, position = 100)
public final class DefaultGreetingService implements GreetingService { … }

GreetingService g = Lookup.getDefault().lookup(GreetingService.class);     // anywhere
```

Per-user services are registered under `Services/Session` (`@Registration(path = "Services/Session", type = PartsModel.class)`) and created once per `UISession` — in its constructor, before any task runs, so do not call `UISession.current()` from the service's constructor; bind it lazily from the first call that reaches you on the executor. Look them up in `UISession.current().getLookup()`; implement `AutoCloseable` to be closed with the session. `@Registration` works on any public class with a public no-arg constructor, or on a public static factory method; `name`, `position`, `attrs = {"key=value"}`, `roles` and `references` are optional. `@FolderRegistration(path, displayName, position)` declares a folder with attributes (a menu, a toolbar). `@Hide({"Menu/Tools/old"})` masks entries of other jars.

Sharing a type between jars is ordinary Java: put it in whichever jar the others depend on (Maven dependency) — the Workbench shares one such type, its `LogMessage`, from `workbench-core`.

Platform services a jar may provide: `UIProvider` (the page), `MainWindowDecorator` (customise each tab's main window), `AuthenticationProvider`/`RoleMapper`/`SessionPolicy` (§5), `HelpCtx.Displayer` (the help system's), `PageProvider` (a full HTML page under a path prefix for signed-in users, rendered on the session executor — the help system's `/wb/help/<id>` is one).

The class path is read once at start-up: `Platform.start` builds the registry from every `RegistrationProvider` the class loader can see and publishes the platform singletons plus every `@ServiceProvider` in `Lookup.getDefault()`. Changing the application means restarting it; services that come and go within a session belong in the session lookup.

5. Users, sessions, roles, per-user data

- `UISession.current().getPrincipal()` → id, name, display name, email, roles, attributes, provider. `principal.hasRole("admin")` for programmatic checks; `Security.checkRole("admin")` throws `AccessDeniedException`; `Security.isAllowed(permission, target)` consults the `Authorizer`s in the lookup.
- Declarative checks: `roles = {"admin"}` on `@TopComponent.Registration`, `@ActionRegistration`, `@OptionsPanelController.TopLevelRegistration`, `@HelpPage` and `@Registration`. Entries not allowed for the user are never sent to the browser: their windows, menu items (including the Window-menu entry of a window), toolbar buttons, shortcuts, popup entries and help pages do not exist for that user, and `findTopComponent` returns null.
- `UISession.current().getUserFiles().dir("com.acme.hello")` → a per-user directory your code owns. Use `userFiles.write(path, bytes)` for durable (atomic) writes.
- `WbPreferences.forModule(Hello.class)` → `java.util.prefs.Preferences` stored under the user's `preferences/<package>.properties` (flat, written through immediately; `Actions.checkbox` menu items and options panels use it). `WbPreferences.forModule(session, Hello.class)` from outside a session task.
- Session-wide events: `session.publish(new OrderPlaced(id))`; `session.subscribe(OrderPlaced.class, e -> …)` — delivered on the session executor to every subscriber of that user, whichever tab subscribed (the Workbench's Output window shows `LogMessage`s published by any window). `subscribe` returns a handle; remove it in `componentClosed`. Cross-user communication goes through a service you write in `Lookup.getDefault()`; `SessionDirectory` (in the default lookup when the server runs) lists every live `UISession`.
- Authentication is pluggable (`design/authentication.md`): implement `org.webbeans.auth.spi.AuthenticationProvider` (`supports`, `authenticate`, `getLoginMethod` — a form, a redirect to an identity provider, or a client script such as the Firebase SDK) and register it with `@ServiceProvider(service = AuthenticationProvider.class, position = 50)` to sit before the shipped properties-file provider. `RoleMapper`s add or translate roles, `SessionPolicy` sets timeouts per user, `AuthenticationListener`s audit logins. Several tabs of one browser share one `UISession`; `UI.logout()` ends it for all of them.
- Multiple tabs: each `GET /` creates a new `UI` with its own component tree and window layout; `session.getUIs()` lists them. Notifications and progress are session-wide and appear in every tab.

6. Internationalisation

- Put strings in `@Messages` (`org.webbeans.core.i18n.Messages`) on the class (or on `package-info.java` for shared keys); call the generated `Bundle.KEY(args)`. Keys with `{0}`… placeholders become methods with parameters. A key may appear once per package; give shared keys one home.
- Any registration attribute starting with `#` is a key in that package's bundle: `displayName = "#CTL_Save"`.
- Translations: `Bundle_de.properties` next to the class (in `src/main/java`, NetBeans style). The processor writes the `@Messages` defaults to `Bundle-generated.properties` in the class output and never touches `Bundle.properties`: a hand-written `Bundle.properties` (keys you reference as `#KEY` from registrations, or share with other code) and the generated file are merged at run time, hand-written files first — so `Bundle_de.properties` translates `@Messages` keys too, a package needs no base `Bundle.properties` for its `Bundle_de.properties` to count, and it does not matter whether your build copies resources before javac (Maven), after it (NetBeans Ant projects) or into a separate directory (Gradle). A key that is both hand-written and in `@Messages` is a compile error. Help pages: `page_de.md` next to `page.md`. Branding: start the launcher with `--branding acme` and put `Bundle_acme.properties` (and `Bundle_acme_de.properties`) next to any bundle you want to override — the application jar is the place for overlays of platform bundles (the Workbench renames the Help menu that way).
- The locale is per session (`UISession.getLocale()`). Generated `Bundle` methods and `Bundles.getMessage(Class, key, args)` use it automatically on the session thread; NetBeans' own `NbBundle.getMessage` is bound to the JVM default locale, so do not use it for user-visible text. Changing the locale (`session.setLocale(Locale.GERMAN)`) relabels the dock, the explorer's menus, the menu bar and the toolbars (rebuilt from the registry) and calls `localeChanged()` on every open window — override it to `setDisplayName(Bundle.CTL_MyWindow())` and refresh your own labels; other components subscribe to `UISession.LocaleChangeEvent` on the session bus. Actions created from registrations resolve their `displayName` on every read, so one shared action instance serves users in different languages.
- Format numbers and dates with `java.text`/`java.time` and `UISession.current().getLocale()`; property editors already do; the browser never formats anything except chart tooltips and number/date fields (from a server-compiled spec).

7. Window system

- A `TopComponent` is a `Panel` that lives in a `Mode` as a tab. Register singletons with `@TopComponent.Registration(mode = …, openAtStartup = …, roles = …)` + `@TopComponent.Description(preferredID = …, iconBase = …, helpId = …)`; add `@TopComponent.OpenActionRegistration(displayName = "#CTL_…")` for a Window-menu entry. Standard modes: `explorer`, `navigator`, `editor`, `properties`, `output` (and the help system's `help`); declare your own with `@Mode.Registration(name, side, defaultSize)` on any class (the Workbench's `admin` mode sits on the window that uses it). Put `webbeans-processor` **and** `webbeans-windows-processor` on the annotation processor path.
- Singleton components: let the platform create them (`openAtStartup`, the generated open action, or `WindowManager.getDefault().findTopComponent("MyTC").requestActive()`).
- Multi-instance components (editors): construct them yourself and call `open()` + `requestActive()`; they get ids `MyTC#1`, `#2`… To have them reopen at the next login, give them `writeState` and a `Factory`: `@TopComponent.Description(preferredID = "EditorTC", factory = EditorTC.Factory.class)` with `public static final class Factory implements TopComponent.Factory { public TopComponent create(Map<String,Object> state) {…} }` (return null when the thing no longer exists). Find an existing instance by walking `wm.getOpened()` (see `EditorTopComponent.openFor` in the Workbench).
- Lifecycle callbacks run on the session executor: `componentOpened` (set the root context, subscribe, start timers), `componentShowing/Hidden`, `componentActivated/Deactivated`, `canClose` (veto; ask asynchronously and call `close()` again), `componentClosed` (unsubscribe, stop timers), `localeChanged` (re-read your strings).
- Tab decoration: `setDisplayName`, `setHtmlDisplayName("<b>…</b>")`, `setIcon(Resource.of(…))`, `setToolTipText`, `setClosable(false)`, `makeBusy(true)` (spinner), `setModified(true)` (marker). Help: `getHelpCtx()` is what `F1` shows while the window is active — the `helpId` of the description, or whatever `setHelpCtx(new HelpCtx("…"))` set (change it as the user moves through the window).
- Each browser tab has its own `WindowManager` and layout; changes are saved per user two seconds after they happen (`layout.json`), and a new tab starts from the most recently saved layout. `wm.saveLayout("name")`/`loadLayout`, `setRole("admin")` for per-role layouts, `resetLayout()` for the registry defaults. Ship a layout with `@LayoutRegistration(role = "admin", resource = "com/acme/app/layouts/admin.json")` (the JSON a saved `layout.json` contains): it is used for users who have no saved layout for that role. `wm.setMaximized/setFloating/setMinimized(tc, …)` do what the dock's buttons do; users can also drag tabs between modes, drop them at a mode's edge (a new split), float them, minimise a mode to the side bar and resize the gutters.
- Put selection into the component's lookup (`associateLookup(ExplorerUtils.createLookup(em, actionMap))`, or `Lookups.fixed(selectedThing)`) — `wm.getGlobalContext()` follows the activated component, and that is what the Properties window, menus, toolbars and shortcuts react to. Put an `ActionMap` into the lookup too: the global Edit menu, its toolbar and `Ctrl+C/X/V`, `Delete`, `F2` are callback actions that delegate to the `copy/cut/paste/delete/rename/find` entries of the activated window's map. `wm.getRegistry().addPropertyChangeListener` tells you when the activation or the set of open windows changes. The **activated nodes** are stickier than the global context, as in NetBeans: `wm.getActivatedNodesContext()` (and

`getRegistry().getActivatedNodes()`, with `PROP_ACTIVATED_NODES`) follow the last activated window that *owns* nodes — an `ExplorerManager.Provider`, or any window whose lookup held a node when it was activated — and stay put when the user clicks a window without nodes. Bind a Properties window to that, not to `getGlobalContext()`, or clicking into the sheet clears it. `setActivatedNodesOwner(false)` on a window that shows nodes without owning them, `true` on a custom window whose lookup may be empty but should still take over when activated.

- Status bar: `StatusDisplayer.getDefault().setStatusText("…")` from any thread; permanent cells are static factory methods registered with `@Registration(path = "StatusBar", type = Component.class)` (one instance per UI). The actions jar contributes the progress cell and the notifications bell.
- The default page is the main window (`MainWindowProvider`); your own `UIProvider` replaces it, and can call `MainWindowProvider.build(ui)` to get the main window plus its own additions. Code that wants to add to the main window — or run something when a tab opens (apply saved preferences, switch the role) — register a `MainWindowDecorator` service.
- Keyboard: `Ctrl+Shift+Enter` maximize/restore, `Alt+W` close, `Alt+Shift+←/→` cycle tabs, `F1` help; application shortcuts come from `@ActionReference(path = "Shortcuts", name = "D-S")`; bind ad-hoc ones with `ui.bindShortcut("Ctrl+Shift+S", "my.save", runnable)`. Plain keys and `Ctrl+C/V/X/A/Z/Y` stay local while a text field has the focus; no shortcut fires while a modal dialog is open (inside a dialog, `F1` presses its Help button).
- One naming trap: inside a `TopComponent` subclass, `Registration` is `TopComponent.Registration`; write `org.webbeans.ui.Registration` for listener handles.

8. Nodes and Explorer — showing data

Use nodes whenever data is hierarchical, selectable or has actions and properties (files, entities, 3D objects, devices):

1. Model it as `Node`s: extend `AbstractNode`, give each node a `Lookup` with the domain object and its capabilities (`Openable`, `Editable`, `Savable`, …), set the display name and icon (`setIconBase`), override `createSheet()` for properties, `setActions(…)` for the popup (use **shared** action instances — multi-selections keep only actions common to all nodes, by identity) or override `getActions` to append the registered ones of a folder (`Actions.forPath("Files/Actions")`, see §9), and `setCanRename/CanDestroy/CanCut`.
2. Produce children lazily with `ChildFactory<T>` (`createKeys` — on a background thread when created with `asynchronous = true`, a “Please wait…” node shows meanwhile — and `createNodeForKey`), wrapped in `Children.create(factory, true)`; or extend `Children.Keys<T>` and call `setKeys` from `addNotify` and from your model’s listener. Call `factory.refresh(false)` when the underlying data changes; nodes for unchanged keys are reused, so expansion and selection survive. Keys need value equality.
3. Show them with a view inside a TopComponent that implements `ExplorerManager.Provider`: `TreeView`, `OutlineView` (tree + property columns, editable in place), `ListView` (the explored folder, double-click to traverse), `PropertySheet` (follows the selection, or `setNodes(…)` for the global context), `NodeComboBox`.
4. `ExplorerUtils.activateActions(em, actionMap, true)` puts the standard copy/cut/paste/delete actions into your `ActionMap`; `associateLookup(ExplorerUtils.createLookup(em, actionMap))` makes the selection and the map visible to the Properties window, to the Edit menu and to actions.

Context menus (node actions + Copy/Cut/Paste/Delete/Rename, shown as a server-side `Popup`), inline rename (F2), delete (Delete key, with a confirmation dialog), in-place editing of writable properties, drag & drop (`setDragSource/setDropTarget`, `getDropType` + `PasteType` on the target), keyboard navigation, quick search and sorting then work with no further code. A selection set from code (`em.setSelectedNodes`) is revealed: the views expand the ancestors of the selected nodes. Expansion is remembered on the server — what the user or `tree.expandNode(n)` expanded is expanded again after a reconnect or a root change, and a node expanded before its parent’s children were sent expands as soon as they arrive.

Properties:

```
set.put(new PropertySupport.ReadWrite<String>("color", String.class, Bundle.PROP_Color(),
Bundle.HINT_Color()) {
    @Override public String getValue() { return String.format("#%06x", part.getColor()); }
    @Override public void setValue(String v) {
        if (!v.matches("#[0-9a-fA-F]{6}")) throw new IllegalArgumentException(Bundle.ERR_Color());
// shown at the cell
        model.recolor(part, Integer.parseInt(v.substring(1), 16));
// the model notifies every view
    }
});
```

The editor is chosen by value type (`PropertyEditors.forType`: text, numbers, boolean, enums as a choice list; register your own with `PropertyEditors.register`) or set per property with `setPropertyEditor`. If several nodes represent one object (a tree node and an editor's node), route changes through your model so each node fires its own change, as the Workbench's `PartsModel` and the demo's `SolarSystem.planetChanged` do.

Persistence: a node that can be found again returns a `Node.Handle` (`getType()` + `toMap()`); a `NodeHandleFactory` registered under `Nodes/Handles/<type>` restores it (`NodeOp.restore`). Editors opened from nodes store what they need in `writeState` and rebuild from it in their `TopComponent.Factory`.

Properties of enum type are shown and edited through `EnumLabels` (`org.webbeans.ui`): the label comes from the enum's `Labeled` implementation, an `EnumLabels.register(...)` function, the bundle key `<SimpleName>.<CONSTANT>` in the enum's package, or the prettified constant name; `PropertyEditors.registerEnum(type, label, icon)` overrides it per type, and any `Node.Property` can draw its value as a `Cell` (`setRenderer`) — icon, html, style and tooltip reach `OutlineView` cells and the Properties window (`getValueAsCell`), while `getValueAsText` stays the editable text. The same labels and icons feed `ComboBox`, `ButtonGroup`, `Table` cells and `PropertyGrid`, so an application defines a label once (Component Guide §4, User's Guide §2.6).

For data that is hierarchical but has no actions or properties of its own, `TreeTable` (§Component Guide 3) flattens a `TreeTableModel` into table rows on the server; `PropertyGrid` edits one plain bean with typed editors. Both avoid the node machinery entirely.

9. Actions, menus, toolbars, shortcuts

An `Action` (`org.webbeans.actions`) has a display name, icon, enabled flag, a value map (`NAME`, `ICON`, `ACCELERATOR`, `SELECTED`, …) with property change events, and `actionPerformed(ActionEvent e)` — `e.getLookup()` is the context it was invoked with (the selection), `e.getSource()` the presenter, `e.getUI()` the tab. `AbstractAction` is the base for hand-written ones; `ActionMap` holds a window's `copy/cut/paste/delete/rename/find` implementations; `ContextAwareAction.createContextAwareInstance(lookup)` binds a selection-sensitive action to one lookup. Registered actions are shared, lazily created instances; presenters bind them to the UI's global context, popups to the selection they were opened on.

You want	Write
Always-enabled action	public class with a no-arg constructor implementing `ActionListener`
Enabled when exactly one `X` is selected	constructor `(X x)`
Enabled when ≥1 `X` selected	constructor `(List<X> xs)`
Delegate to the active window's `ActionMap` (`copy`, `delete`, your own key)	`@ActionRegistration(…, key = "delete")`; an abstract class means "no fallback" (disabled without a map entry)
Checkbox menu item backed by a user preference	`Actions.checkbox(Module.class, "key", default, onToggle, label)` from a `Presenter.Menu` action (see the Workbench's `SpinAction`)
Full control (own presenter, own enablement)	a public `Action` subclass with a no-arg constructor; implement `Presenter.Menu/Toolbar/Popup` to supply components
Background work	`@ActionRegistration(…, asynchronous = true)`: runs on a `RequestProcessor` with a "Running …" progress handle; use `e.getUI().access(…)` for components
Menu / toolbar / popup placement	`@ActionReference(path = "Menu/File", position = …, separatorBefore/After = …)`, `"Toolbars/File"`, `"<Type>/Actions"` (any folder a node reads with `Actions.forPath`), `Scene3DTopComponent.ACTIONS_FOLDER`, `ChartTopComponent.ACTIONS_FOLDER`, `GraphTopComponent.ACTIONS_FOLDER`, `MapTopComponent.ACTIONS_FOLDER`
Keyboard shortcut	`@ActionReference(path = "Shortcuts", name = "D-S")` (`D` = Ctrl/Cmd, `O` = Alt, `S` = Shift, keys `S`, `F5`, `ENTER`, `DELETE`, `UP`…)
A menu or toolbar of your own	`@FolderRegistration(path = "Toolbars/File", displayName = "#CTL_FileToolbar", position = 100)` on any class
Keep the last selection when a window without one is activated	`surviveFocusChange = true` (the Workbench's Save stays enabled while you look at the Files window)
Several small actions in one file	nested `public static final class` actions inside a holder class

```
@ActionID(category = "File", id = "org.webbeans.workbench.files.SaveAction")
@ActionRegistration(displayName = "#CTL_Save", iconBase =
"org/webbeans/workbench/files/icons/save.svg", surviveFocusChange = true)
@ActionReferences({
    @ActionReference(path = "Menu/File", position = 300),
    @ActionReference(path = "Toolbars/File", position = 300),
    @ActionReference(path = "Shortcuts", name = "D-S")})
public static final class SaveAction implements ActionListener {
    private final Savable savable;
    public SaveAction(Savable savable) { this.savable = savable; }         // exactly one Savable
in the context → enabled
    public void actionPerformed(ActionEvent e) {
        try { savable.save(); } catch (IOException ex) { DialogDisplayer.getDefault().notify(new
NotifyDescriptor.Exception(ex)); }
    }
}
```

What the platform does with it: the entry `Actions/File/org-webbeans-workbench-files-SaveAction` (attributes: display name, icon, delegate class, context type `Savable`, selection `EXACTLY_ONE`), references in the menu, the toolbar and the shortcut map. At login the main window gets a `MenuBar` whose top-level menus are the `Menu/*` folders in position order (a declared but empty menu is not shown), filled when first opened; a `ToolbarPanel` with one `Toolbar` per `Toolbars/*` folder; and the `Shortcuts/*` keys bound on the tab. Every item is bound to the action: enabled follows the selection (a `Lookup.Result` on the global context), the accelerator is shown, choosing it invokes the action with the global context. Menus and toolbars are per user (roles, locale, customisation) — you never build a `MenuBar` yourself, though you can (`MenuBarBuilder.build(context)`, `ToolbarBuilder.build(context)`, `ActionPresenters.menuItem(action, context)`). `View ▸ Toolbars` hides toolbars through the user's registry overlay (`users/<u>/config/registry.json`).

Programmatic use: `Actions.forID("File", "org.webbeans.workbench.files.SaveAction")`, `Actions.forPath("Files/Actions")` (null = separator), `Actions.invoke(action, source, context)`, `Actions.actionsToPopup(actions, context, x, y)` (what node views show on right-click; the `Popup` lives in the overlay and removes itself when dismissed), `Actions.context(...)`, `Actions.alwaysEnabled(...)`, `Actions.callback(...)`.

Async pattern for a confirm-before-close:

```
@Override protected boolean canClose() {
    if (!dirty) return true;
    DialogDisplayer.getDefault().notify(new
NotifyDescriptor.Confirmation(Bundle.MSG_SaveChanges(name),
NotifyDescriptor.YES_NO_CANCEL_OPTION))
        .thenAccept(r -> { if (r == NotifyDescriptor.YES_OPTION) { save(); close(); } else if (r
== NotifyDescriptor.NO_OPTION) { dirty = false; close(); } });
    return false;                                       // keep open now; close() again once nothing
is dirty
}
```

Quick Search (`org.webbeans.quicksearch`): Edit ▸ Quick Search… / Ctrl+I opens a box over the main window; results come from `QuickSearchProvider`s registered under `QuickSearch/Providers` (`@Registration(path = QuickSearch.PROVIDERS, type = QuickSearchProvider.class)`): the platform's find menu actions by name or initials (run with the global context), open windows, and help topics; each provider's `search(query, response)` runs on the session thread per debounced

keystroke and adds up to eight `(displayName, detail, icon, Runnable)` results. One `QuickSearch` component per UI lives in the root’s overlay slot and keeps working while a modal dialog is open.

10. Dialogs, notifications, progress, options, help

- Simple messages: `DialogDisplayer.getDefault().notify(new NotifyDescriptor.Message(text, NotifyDescriptor.INFORMATION_MESSAGE))` (the message may be a component, e.g. an `Html`); questions: `NotifyDescriptor.Confirmation(text, title, YES_NO_OPTION)`; one text field: `NotifyDescriptor.InputLine(label, title)` → `getInputText()` after `OK_OPTION`; errors with a stack trace: `NotifyDescriptor.Exception(throwable)`. The future completes on the session executor with the option chosen, or `CLOSED_OPTION` for Escape / the close button. Dialogs are modal by default: the page is darkened, keys and clicks outside are ignored, and the server drops events from outside the dialog (a stale client cannot bypass it). `descriptor.setHelpCtx(new HelpCtx("…"))` adds a *Help* button at the left of the button bar (`F1` presses it); set it again at any time to change or remove the button.
- Forms: build a `Panel`, wrap it in a `DialogDescriptor(content, title, modal, optionType, initialValue, buttonListener)` (or give explicit options: standard constants, strings, your own `Button`s; the NetBeans-shaped constructor takes a `HelpCtx` too), read the values in `thenAccept`. `setValid(false)` disables OK/Yes/the default button until the form validates; `createNotificationLineSupport().setErrorMessage(…)` shows a line under the content; `setClosingOptions` keeps the dialog open for an “Apply” button while the `buttonListener` sees every click (`e.getActionCommand()` is the button’s label); `createDialog(descriptor)` returns a `Dialog` to `show()`/`hide()`/`setTitle` yourself. `Enter` presses the default button, the title bar drags, focus is trapped inside.
- Wizards: `WizardDescriptor` over `Panel`s (`AbstractPanel` gives change support): each step has a component, `isValid()` (drives Next/Finish), `readSettings`/`storeSettings` on a shared settings object; the step list sits on the left, `PROP_ERROR_MESSAGE` shows under the content; an `InstantiatingIterator` runs on Finish. Minimal by design: a fixed `ArrayIterator`, no asynchronous instantiation yet.
- Toasts: `NotificationDisplayer.getDefault().notify(title, icon, details, onClick, priority, category)` (or the short `notify(title, details, category)`) — reaches every tab of the user (toast + the drawer behind the status bar bell), queued in the drawer if no tab is open; `HIGH` toasts stay until dismissed, `SILENT` ones go to the drawer only. `notification.clear()` removes it everywhere. Safe from any thread; from a future’s continuation use `NotificationDisplayer.getDefault(session)`.
- Status bar: `StatusDisplayer.getDefault().setStatusText(...)`; progress: `ProgressHandle h = ProgressHandleFactory.createHandle(name, cancellable); h.start(100); h.progress("step", 40); h.finish();` — safe from any thread; the newest running handle shows in the status bar (bar, text, a cancel button when a `Cancellable` was given, a list of all running tasks on click), updates coalesced to ten per second.
- **Options** (Tools ▸ Options): one category per settings page.

```
@OptionsPanelController.TopLevelRegistration(categoryName = "#CTL_DashboardOptions", iconBase =
"…/chart.svg", position = 200)
public final class DashboardOptions extends OptionsPanelController {
    private final TextField refresh = new TextField();
    private Preferences prefs;
    public Component getComponent(Lookup master) {                    // once per dialog
        prefs = WbPreferences.forModule(DashboardOptions.class);
        refresh.setValidator(v -> inRange(v, 1, 60) ? null : Bundle.ERR_Range(1, 60));
        refresh.addChangeListener(e -> changed());                     // enables Apply, re-
checks isValid → OK
        Panel p = new Panel(new Layout.Form()); p.addRow(Bundle.LBL_Refresh(), refresh); return p;
    }
    public void update()       { refresh.setValue(String.valueOf(prefs.getInt("refreshSeconds",
1))); }
    public void applyChanges() { prefs.putInt("refreshSeconds",
Integer.parseInt(refresh.getValue().trim())); DashboardTopComponent.settingsChanged(); }
    public void cancel() {}
    public boolean isValid()   { return inRange(refresh.getValue(), 1, 60); }
    public boolean isChanged() { return !String.valueOf(prefs.getInt("refreshSeconds",
1)).equals(refresh.getValue()); }
    public String getHelpId()  { return "dashboard.options"; }          // the dialog's Help
button while this category is selected
}
```

`OptionsDisplayer.getDefault().open("DashboardOptions")` opens the dialog on a category and completes with `true` on OK. Categories carry `roles`; the dialog creates a fresh controller per opening, calls `update()` before showing a page, `applyChanges()` on OK/Apply for changed controllers and `cancel()` on Cancel.

- **Help** (`F1`, Help ▸ Help Contents, the Help buttons of dialogs): pages are Markdown or HTML resources in your jar, registered with `@HelpPage` on a class or `package-info.java`:

```
@HelpPage(id = "files", parent = "workbench", title = "#HLP_Files", resource =
"org/webbeans/workbench/files/help/files.md", position = 100, keywords = "#KW_Files")
@HelpPage(id = "files.editor", parent = "files", title = "#HLP_Editor", resource =
"org/webbeans/workbench/files/help/editor.md", position = 100)
@Messages({"HLP_Files=Files", "KW_Files=explorer, folder, rename, delete", "HLP_Editor=The
editor"})
package org.webbeans.workbench.files;
```

The table of contents is a tree over the registry folder `Help/Pages` (`parent` nests pages — across jars, too; `position` orders; `roles` filters; the first top-level page is the **home page**). Connect pages to the UI with `helpId` on `@TopComponent.Description` (or `setHelpCtx`), `NotifyDescriptor.setHelpCtx`, `OptionsPanelController.getHelpId`, or `new HelpCtx("files.editor").display()` from code. In a page, link to other pages by file (`[editor](editor.md)`, `../../core/help/workbench.md`) — resolved to the registered page — or with `help:<id>` and `#heading-slug`; images next to the page are served from the jar; `page_de.md` is the German variant. The Help window (`HelpTC`, its own mode on the right) shows breadcrumbs, the page, a search over titles, keywords and text, Back, and *Open in new tab*, which leads to `GET /wb/help/<id>` — the same page as a stand-alone document for bookmarks and printing (signed-in users, same filtering). The platform ships one page, `platform` ("Getting around"), about the window system; link to it from your overview. Under the hood: `HelpSet.current()` (the topics), `HelpContent.html(set, topic, locale)` (rendering), `Markdown.toHtml`, `HelpViewer` (the component), `HelpDisplayer` (the `HelpCtx.Displayer` service), `HelpPageProvider` (the `PageProvider`).

11. Charts

A `Chart` is an ordinary component whose data lives on the server: you mutate the `ChartModel`, the platform sends the difference. Put one anywhere a component goes (a panel, a `TopComponent`, a dialog); `ChartTopComponent` gives you a window with a toolbar (reset zoom, save as SVG/PNG, plus the actions registered under `Charts/Actions`).

```
Chart cpu = new Chart(ChartType.LINE);
cpu.setTitle(Bundle.CHART_Cpu()).setZoom(ZoomMode.X).setTooltip(TooltipMode.SHARED);
cpu.getXAxis().setType(AxisType.TIME).setFormat("HH:mm:ss");
cpu.getYAxis().setMin(0.0).setMax(100.0).setFormat("#0'%'").addBand("hot", 90, 100, "#b43b3b");
Series s = cpu.getModel().addSeries("cpu", Bundle.SER_Cpu());
s.setWindow(120);                                                    // sliding window
on both sides
timer = ui.scheduleAtFixedRate(() -> s.addPoint(Point.of(Instant.now(), sampler.cpu())), 0, 1000);
cpu.addPointClickListener(e ->
StatusDisplayer.getDefault().setStatusText(String.valueOf(e.getPoint())));
```

- **Types**: `LINE`, `STEP`, `AREA`, `BAR`, `SCATTER` (bubbles with `withSize`), `PIE`, `DONUT`; the chart has a default and each series may override it where the axes allow (`LINE` + `BAR`, an `AREA` on the secondary axis via `setYAxis(1)`). `setStacked(true)` or `Series.setStack("group")` stacks bars and areas; `setHorizontal(true)` turns bars sideways; `setLabels(true)` prints values.
- **Points**: `Point.of(x, y)`, `Point.of(instant, y)`, `Point.of("Mars", y)` on a category axis (unknown categories are appended). Add `withKey` when you will `updatePoint`/`removePoint` later or want `PointEvent.getPoint()` to find the point reliably; `withLabel`, `withColor`, `withMeta` for display.
- **Mutations are deltas**: `setPoints` replaces (one call), `addPoint(s)` appends (with `trim` when a window is set), keyed `updatePoint`/`removePoint`, `removeSeries`. Everything you do in one task becomes one `data` call; a chart with 60 points and one append per second costs about 190 bytes per second on the wire.
- **Axes**: `setType` (`LINEAR`, `LOG`, `TIME`, `CATEGORY`), `setMin/Max`, `setFormat` with a `DecimalFormat` pattern (`#,##0.0`, `#0'%'`, `0.00 '€'`) or, for time axes, a `DateTimeFormatter` pattern — compiled per session locale, so German users see `1.234,5`; `addMarker` (reference line) and `addBand` (shaded range); `setGrid`, `setTitle`.
- **Interaction**: tooltips (`POINT`, `SHARED` = every series at that x, `AXIS`, `NONE`), legend toggling (`LegendEvent`, `Series.isVisible()` follows), `setZoom(ZoomMode.XY)` for wheel zoom, drag pan and shift-drag brush (`RangeEvent`, `isReset()` on double-click/Escape), `setSelectionMode` for click highlighting, `highlight(seriesId, keyOrX)` and `setXRange(from, to)` from the server (link charts by forwarding one chart's `RangeEvent` into another's `setXRange`).
- **Export**: `exportPng(w, h)` / `exportSvg()` render in the browser and complete on the session executor (`thenAccept`); `download("png" | "svg", fileName)` saves straight from the browser.
- Stop timers in `componentClosed`; the model survives, so reopening the window shows the data again (the chart re-sends everything when attached or after a reconnect).

12. 3D scenes

A `Scene3D` is a component whose scene graph lives on the server: you build `Node3D`s under `scene.getRoot()`, the platform sends every change as a small op (merged per task into one `scene` call), and the browser renders with three.js — downloaded only by tabs that open a 3D view. `Scene3DTopComponent` gives you a window with a toolbar (fit, reset view, grid, save PNG, plus the actions registered under `Scene3D/Actions`).

```
Scene3D scene = new Scene3D();
scene.setGrid(40, 40).setHoverEffect(HoverEffect.HIGHLIGHT).setBackground(0x0b1020);
scene.getCamera().setPosition(0, 18, 26).lookAt(Vector3.ZERO);
scene.getControls().setDistanceLimits(4, 120).setPolarLimits(0.05, Math.PI / 2 + 0.3);
scene.getRoot().add(new AmbientLight(0xffffff, 0.35), new PointLight(0xfff2cc, 1.6));
Material gold = Material.standard(0xffcc33).setEmissive(0xff9900, 1.2);                    //
shared by id
Mesh sun = scene.getRoot().addChild(new Mesh("Sun", Geometry.sphere(1.8, 48, 24), gold));
sun.animate(Animation.rotate(Axis.Y, 0.15));                                                //
runs in the browser
scene.addListener(Scene3D.PICK, e -> { scene.setSelection(e.getNode()); status(e.getUserData());
});
```

- **Nodes**: `Group` (transforms compose), `Mesh` (`Geometry` + `Material`), `InstancedMesh` (one node for thousands of copies; `setInstanceTransform/Color`, picks report the instance), `Line` (`Line.circle` for orbits), `Points`, `Label3D` (a DOM label that follows a point; `setOccludable`), lights (`AmbientLight`, `HemisphereLight`, `DirectionalLight`, `PointLight`, `SpotLight`). Every node has position/rotation (`Quaternion`, or `setRotationEuler`)/scale, `setVisible`, `setPickable`, shadows, `setUserData(domainObject)` — the user data is in the node's lookup and in every event — and `getLookup()` to override.
- **Geometry** is immutable and content-addressed: primitives (`box`, `sphere`, `plane`, `cylinder`, `cone`, `torus`, `circle`, `ring`) travel as parameters; `Geometry.buffer().positions(…).normals(…).uvs(…).indices(…).build()` (the Workbench's turbine blade) and `Gltf.load(stream)` (GLB / embedded glTF 2.0: hierarchy, materials, textures) become binary blobs served once from `/wb/blob/<sha256>` and cached by the browser. Build big geometry off-thread; attach it on the executor.
- **Materials** are mutable and shared: `Material.standard(rgb)`, `basic`, `phong`, `lambert`, `wireframe`; `setColor/Opacity/Metalness/Roughness/Emissive/Map/NormalMap/Side/FlatShading/DepthTest`. Changing one is a single update for every mesh and scene using it. `LineMaterial` (width, dashes) and `PointsMaterial` (size) for lines and point clouds; `Texture.image(bytes, type)` / `Texture.fromResource(path)` with `repeat/wrap/filter/flipY`.
- **Camera and controls**: `PerspectiveCamera`/`OrthographicCamera` with `setPosition`, `lookAt`, `setClipping`; `Controls.orbit()` (limits, auto-rotate, `setLocked`), `map()`, `firstPerson(speed)`, `none()`. `scene.fit()`/`fit(node)` frames, `flyTo(state, ms)` animates, `getCameraState()` is what the user last left (updated by `CAMERA_CHANGE` events before listeners run).
- **Events**: `PICK` (node or null for the background, hit point and normal, instance, button, double-click, modifiers; `e.getLookup()` is the node's), `HOVER` (throttled; sent only when listened — `setHoverEffect` works without traffic), `DRAG`, `CAMERA_CHANGE`, `ANIMATION_DONE`, `READY` (`webgl2`, renderer name).

- **Drags**: `node.setDraggable(DragConstraint.plane(Axis.Y, 0).snap(0.5))` (or `axis(Axis.X)`, `free()`). The browser moves the node smoothly; the server re-validates each report against the constraint, applies it to the node, then calls your `DRAG` listener (`START`/`MOVE`/`END`, `getPosition()` validated, `getStart()`); call `e.reject()` to snap it back. Non-draggable nodes can never be moved from the browser.
- **Animations**: `Animation.rotate(axis, radPerSec)`, `orbit(center, axis, rate)`, `moveTo(to, ms)`, `scaleTo`, `path(points, ms)` with `loop/yoyo/delay/easing/onComplete`; `node.animate(a)`, `a.pause()/resume()/seek()/stop()`. Timed animations report their final transform once; the node follows and `onComplete` runs on the executor. Running animations survive reconnects.
- **Binding data**: `new SceneBinder<T>(group, keyFn, createFn, updateFn).setItems(collection)` diffs by key — new items get nodes (user data set to the item), existing ones are updated in place, missing ones removed; node setters are no-ops for unchanged values, so a naive `update` costs nothing on the wire. `refresh(item)` after an in-place change (the Workbench's sensor readings).
- **Inspector for free**: `em.setRootContext(new AbstractNode(new Scene3DChildren(scene.getRoot())))` shows the graph in a `TreeView` (name, type, visible, x/y/z, scale as editable properties; rename goes to the node); `Scene3DChildren.bindSelection(em, scene)` syncs picking and tree selection both ways, so the Properties window edits picked objects. The Workbench instead shows its own domain nodes (`PartNode`) and syncs through the model — use whichever your data suggests.
- **Pictures**: `scene.snapshot(w, h)` completes with PNG bytes on the executor; `scene.download("scene.png", 0, 0)` saves straight from the browser. `setStats(true)` shows fps/draw calls while tuning.
- Colours are `0xRRGGBB` ints; angles are radians; one unit is whatever your model says (keep scenes within ~1e4 units of the origin for float precision).

12a. Graph scenes

`GraphScene<N, E>` (`org.webbeans.graph`) is the Visual Library idea: your objects are mapped to widgets, and the widgets say how they look — shape, pins, accent, icon, anchors, router, end shapes, labels — per object, in `attachNodeWidget`/`attachEdgeWidget` overrides or on the widget `addNode`/`addEdge` returns. Notes are free text attached to any widget. Sizes are measured by the browser, so server-side layouts (`TreeGraphLayout`, `GridGraphLayout`, `CircularGraphLayout`, `ForceGraphLayout`) place nodes exactly, animated. The user selects, drags, connects (a `ConnectProvider` decides and creates), renames in place, right-clicks (a `PopupProvider`, or `GraphScene.popupFromRegistry("MyType/Actions")` for registered context actions) and presses Delete; every change is a small delta merged per flush. `GraphTopComponent` wraps a scene with a toolbar and puts the selection (objects and widgets) in the window's lookup, so the Properties window and context actions follow it. The component guide §7a has the full API; the demo's *Deployment graph* window (`DeploymentTopComponent`) uses most of it.

12b. Maps

`MapView` (`org.webbeans.map`) is a slippy map whose overlays live on the server — markers, lines, polygons, circles in metres, rectangles, georeferenced images, GeoJSON, popups, grouped into layers the user toggles — with the browser reporting view changes, clicks (with the overlay or GeoJSON feature hit), hovers, marker drags and finished drawings (the five draw modes). Tiles are served **through the server**: `TileSource.OSM` is proxied and cached on disk (`/wb/tiles/…`, a `ContentProvider`), which keeps the Content Security Policy strict and works on intranets whose browsers have no internet route; `TileSource.directory` serves a local tile folder for air-gapped sites, `TileSource.direct` loads from the provider when you allow its origin with `ServerConfig.imageSources`, and `TileSource.BLANK` draws a graticule. `MapTopComponent` adds the toolbar (zoom, show everything, draw tools, `Map/Actions`). Component guide §7b has the API; the demo's *Sites map* window (`SitesMapTopComponent`) uses most of it.

`ContentProvider` (`org.webbeans.ui`) is the general mechanism behind the tile endpoint: a binary counterpart of `PageProvider`, served under a prefix to signed-in users on the HTTP thread (thread-safe, may block) — register one with `@ServiceProvider(service = ContentProvider.class)` for any bytes your components need to load by URL.

13. Writing your own component

When a built-in component does not fit, write one: a Java class and a JavaScript renderer.

```
@Renderer(id = "acme.Gauge", resource = "com/acme/widgets/gauge.js", css =
"com/acme/widgets/gauge.css")   // resources in the jar; bundled at startup
public class Gauge extends Component {
    public static final Property<Double> VALUE = Property.of("value", Double.class, 0.0);
    public static final Property<Double> MAX   = Property.of("max",   Double.class, 100.0);
    public static final EventType<ClickEvent> CLICK = EventType.of("click", ClickEvent::new);
    public Gauge() { super("acme.Gauge"); }
    public Gauge setValue(double v) { set(VALUE, v); return this; }
    public Registration addClickListener(Listener<ClickEvent> l) { return addListener(CLICK, l); }
    public void flash() { call("flash"); }
}
```

```
// gauge.js — plain script concatenated into the bundle; CSS variables for theming; emit only
semantic events
wb.register('acme.Gauge', {
  create(ctx) { const el = wb.h('div', 'acme-gauge');
                el.innerHTML = '<svg viewBox="0 0 100 60"><path class="acme-gauge-arc"/></svg>';
                el.addEventListener('click', () => ctx.emit('click', {})); return el; },
  update(el, props, changed) { if (changed.has('value') || changed.has('max')) draw(el,
props.value / props.max); },
  call(el, method, args) { if (method === 'flash') el.animate([{opacity: 0.3}, {opacity: 1}],
300); },
  insertChild() {}, removeChild() {}, destroy(el) {}
});
```

Guidelines: keep state on the server unless it is purely visual; mark properties the browser owns (`Property.clientOwned()`) and continuous events `lazy(ms)`; for a value-bearing input extend `ValueComponent<T>` (validation, `required`, `readOnly`, `change` events come for free); never put HTML from users into `innerHTML` (use text nodes, or run it through `Html.sanitize` on the server as the help viewer does); style with `--wb-*` CSS variables and prefix your own classes (the platform owns `wb-`); set `setTestId("...")` on components your tests need (`data-test` attribute, `HeadlessUI.byTestId`). If the component delivers bulk state with `call(...)` rather than properties (the explorer views, the notification host, the chart and the 3D scene do), override `resync()` to send it again — the platform calls it when a tab gets a full `init`. To coalesce many mutations of one task into a single call, register `getUI().beforeFlush(runnable)` on the first mutation and emit the merged call there (`Chart` and `Scene3D` are the pattern). Make calls from `attached()`, never from the constructor's expectations about the wire. A transient component in the overlay (a popup, a toast) that must keep working while a modal dialog is open registers with `ui.allowWhileModal(component)`. Large binary payloads (geometry, images) go through `BlobStore.getDefault().put(bytes, type)` and are fetched by the renderer from `/wb/blob/<id>` (content-addressed, cacheable). A renderer that needs a third-party library loads it on demand with a script tag from `/wb/res/` (as `scene3d.js` does with three.js) rather than bundling it for every page. Code that needs a whole page outside the single-page application (a printable report, a help page) implements `PageProvider` (`@ServiceProvider`): the front end routes its prefix to it for signed-in users and renders on the session executor. Run with `--dev` so the bundle is rebuilt on every page load while you edit the renderer.

More patterns from the step-12 components, all in `webbeans-ui`:

- **Defaults are not sent.** A property keeps its Java default without a `set` op, so a renderer must treat a missing prop as the default (`props.grid == null ? 10 : props.grid`, `props.enabled !== false`) — `ColorPicker`'s palette and `Diagram`'s grid snap both carry a client-side copy of the default.
- **Batched calls.** `OutputView.append` collects lines and registers one `beforeFlush` task per flush that emits a single `append` call; `clear()` is folded into the same task. `resync()` re-sends the ring buffer as one `set`. `Diagram` and `PropertyGrid` coalesce model changes into one property update the same way (`republish`).
- **Resending an unchanged value.** `set(property, sameValue)` sends nothing; when the client must be told again (a rejected `PropertyGrid` edit, a parse failure in a value component), call `resend(property)`.
- **Server-side events.** `fireEvent(type, data)` dispatches an event to the registered listeners from server code — `Kanban`, `Diagram` and `TreeTable` use it to report a drop they applied, with the same event classes a client event would produce. Only listened types are dispatched.
- **Extending a renderer.** A subclass with its own renderer id (`Table(String rendererId)` → `TreeTable`, `Panel(String, Layout)` → `Card`) extends the base renderer **lazily** on the client, because the bundle's order is not fixed: `Object.setPrototypeOf(impl, wb.renderers['wb.Table'])` in `create`, then `Object.getPrototypeOf(impl).create.call(this, ctx)`. `Table.decorateRow(row, encoded)` is the server hook that lets `TreeTable` add depth and expander flags to each encoded row; `Panel` keeps renderer-owned elements marked `wb-panel-keep` across relayouts (the busy overlay, the card header).
- **Shared popups.** `wb.placePopup(el, anchor, placement)` positions a fixed element next to an anchor and flips it when there is no room; elements with `wb-popup-open` close on Escape or an outside click (`wb-popup-sticky` opts out). `SplitButton`, `ColorPicker` and `Popover` share it.
- **Locale on the client.** The `init` message's locale is kept in `wb.locale` (and `<html lang>`), so renderers format dates and numbers with `Intl` for the session locale (`Calendar`'s month and day names).
- **Drag and drop between components.** `Palette` items carry the MIME type `text/x-wb-palette`; any renderer that accepts it (`Diagram`) turns the drop into an event with its own coordinates. `Kanban` and `Calendar` move their own elements with the same HTML5 API and report the result.
- **Text utilities.** `org.webbeans.ui.text.Markdown` (moved here from the help system; `toHtml`, `inline`, `escape`) and `LineDiff` (Myers' diff, `unified`) are plain classes usable without components.

14. Testing

- **`HeadlessUI`** (`webbeans-test`) runs a complete session without a server or browser: the registry from the test class path (every jar on it contributes its registrations), the real main window, a temporary data directory, and a recorder for everything the browser would receive.

```
try (HeadlessUI h = HeadlessUI.builder().user("alice", "user", "admin").start()) {
    TopComponent files = h.open("FilesTC");                                    // open + activate a
registered window
    h.run(() -> Actions.invoke(Actions.forID("File", "…NewFileAction"), null, h.globalContext()));
    assertTrue(h.hasDialog());
    h.type("dialog-input", "todo.txt");                                        // commit a value
into a component by test id
    h.answerDialog("dialog-ok");                                               // press a dialog
button

assertTrue(Files.exists(h.userDir("org.webbeans.workbench.files").resolve("workdir/todo.txt")));
    h.inject(scene, "pick", HeadlessUI.map("node", id, "point", Arrays.asList(0.0, 1.0, 0.0)));
// any client event
    List<Map<String, Object>> calls = h.callsOf(chart, "data");                // what went over
the wire
}
```

Everything that touches components runs through `h.run(...)` (the session executor); the registry is role-filtered there, so role tests belong inside it. `h.openTab("u2")` gives a second tab; `h.messages(tab)` its wire. Dialogs are real: `hasDialog()`, `dialogButtons()`, `answerDialog("dialog-yes")`. `HeadlessUI.builder().dataDir(path)` reuses a data directory, so a second session restores what the first one saved (layouts, preferences).

- Unit-test nodes, models and actions without any UI (`NodesTest` in `webbeans-nodes`: key diffs, factories, sheets, actions; `ActionsTest` in `webbeans-actions`: the action kinds over an `InstanceContent` lookup).
- Lower-level: a `UI` with a recording `Transport` (`UITest`, `ExplorerTest`, `ChartTest`, `Scene3DTest`); window system tests with a `Registry` built from a `RegistrationBuilder` (`WindowManagerTest`); dialogs answered by clicking their buttons (`DialogsTest`); the help system end to end on `HeadlessUI` (`HelpTest`: registry → table of contents, display, F1, dialog buttons, search, the stand-alone page, re-expansion after a restore).
- `MockServices.setServices(FakeFileService.class)` swaps services in `Lookup.getDefault()` for a test; `WbDefaultLookup.getInstance().setContent(...)` replaces the whole default lookup (HeadlessUI restores it on close).
- Registry and platform tests can build real jars at test time (`TestJars` in `webbeans-core`'s tests), put them in a `URLClassLoader` and run `Platform.start` against it and a temp data directory.
- Server tests: `WbServerTest` starts the real Jetty server on port 0 and speaks the protocol over long polling (`HttpURLConnection`) and over a WebSocket (Jetty's `WebSocketClient`); `webbeans-demo/e2e/*.js` (`transport.js` covers the WebSocket, the outage/replay path and the long-poll fallback; `graph.js` the graph scene, `map.js` the map), `webbeans-tutorial/e2e/tutorial.js`, `workbench/e2e/workbench.js` and `workbench/e2e/help.js` drive Chromium with Playwright (headless WebGL through SwiftShader) through the dock, the explorer, the menus, dialogs, charts, 3D, options, help, persistence, reconnect and roles.

15. Running, configuring, packaging

Run the launcher with the platform jars and your application jars on the class path — that is the whole deployment model:

```
java -cp 'lib/*' org.webbeans.launcher.Main --datadir data --port 8080
```

Flag	Default	Meaning
`--port`, `--host`	8080, 0.0.0.0	HTTP endpoint of the embedded Jetty 9.4 server
`--https-port`	off	HTTPS endpoint, TLS terminated by Jetty; the certificate comes from `--keystore <file>` + `--keystore-password` (PKCS12 or JKS by extension, `--keystore-type` overrides; `--key-password` when the key's password differs; `WEBBEANS_KEYSTORE_PASSWORD` works too). Without `--keystore` a self-signed certificate for `localhost` is generated into `<datadir>/system/selfsigned.p12` (development only). HTTPS responses carry `Strict-Transport-Security` (`--no-hsts` turns it off) and `Secure` cookies
`--https-redirect`, `--no-http`	off	with `--https-port`: redirect plain requests to the HTTPS port, or do not open the plain port at all
`--secure-cookies`, `--trusted-proxy <ip>`	off	behind a TLS-terminating proxy: mark cookies `Secure` and believe `X-Forwarded-For/-Proto/-Host` from that address
`--session-store <dir>`, `--node-id <name>`	off, host name	share authenticated sessions between instances through a directory (`FileSessionStore`), so a restart or a failover to another node keeps users signed in; the node id appears in logs, session files and `/wb/health`
`--datadir`	`./data`	users (`system/users.properties`), per-user files, layouts, preferences, registry overlay, cache
`--branding`	none	bundle overlay name: `Bundle_<name>[_<locale>].properties` next to any bundle wins
`--context`	`/`	serve under a path prefix
`--title`	WebBeans	application title (login page, bootstrap page)
`--locale`	JVM default	fallback locale (the browser's `Accept-Language` wins per session)
`--session-timeout`	30m	idle timeout (a `SessionPolicy` in the lookup overrides per user)
`--dev`	off	rebuild the client bundle on every page load, no cache headers
`--admin-password`, `--no-default-admin`	generated	first-start `admin` account
`--no-websocket`	off	long polling only: the page does not try the WebSocket (a proxy that cannot forward the upgrade)
`--http-threads`	4 × cores, min 8	Jetty's thread pool (connections, long polls, socket callbacks; session work runs on its own pool)
`--max-upload`	256m	largest file `/wb/upload` accepts (`512k`, `10m`, `2g`)

Application jar: one runnable jar with the platform, your jars and your branding resources merged. `workbench/workbench-app/pom.xml` builds it with `maven-shade-plugin` (depends on the application jars, merges everything, concatenates `META-INF/services`, sets `Main-Class`);

`tools/fatjar.py` does the same offline. Deploy with `java -jar myapp.jar --datadir /var/lib/myapp --branding myapp --https-port 443 --keystore /etc/myapp/server.p12 --keystore-password … --https-redirect` (Jetty terminates TLS; convert a PEM certificate and key with `openssl pkcs12 -export -in cert.pem -inkey key.pem -out server.p12`), or behind a TLS-terminating proxy with `--trusted-proxy`. The jar is stateless apart from `--datadir`; the browser bundle hash changes with every build, so clients reload automatically after an upgrade.

Several instances / load balancing: sessions are stateful (a user's windows live in the JVM they signed in on), so balance *sticky per session* — a reverse proxy with cookie affinity on `WBSESSION`, WebSocket upgrades forwarded, `--trusted-proxy` on every instance — with the data dir shared (layouts, accounts) and `--session-store` on a shared directory so a failover keeps users signed in (they get a fresh tab, not their open editors). `GET /wb/health` is the balancer's health check. `design/authentication.md` §14 has the full discussion with HAProxy, nginx and Kubernetes configurations, and §8a the `SessionStore` SPI (JDBC/Redis stores are the same five methods as the shipped `FileSessionStore`).

Transport and proxies: the browser opens one WebSocket per tab (`/wb/ws`), keeps it alive with a ping every 30 s and reconnects with backoff, replaying the deltas it missed (a tab survives a network blip or a server-side socket timeout without a reload; after two minutes without a connection its UI is closed and the next connect starts fresh). If the first WebSocket attempt never connects, the tab falls back to long polling (`/wb/poll`) by itself — so the application works behind any proxy, just with higher latency. Behind nginx make the upgrade work: `proxy_http_version 1.1; proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade"; proxy_read_timeout 120s;` (Apache: `mod_proxy_wstunnel`). `document.documentElement.dataset.wbState` is `connected`/`reconnecting`/`gone` and `wb.transport.kind` is `ws` or `poll` — handy in the browser console when debugging a deployment. An application that needs more of Jetty (its own servlets, another connector) gets the `org.eclipse.jetty.server.Server` from `WbServer.getJetty()` before `start()` and configures it; the platform's own servlets live in one `ServletContextHandler` at `--context`.

Files: `Upload` components post to `/wb/upload` (limit `--max-upload`, default 256 MB, files staged under `<datadir>/cache/uploads/`) and `DownloadService` streams from `/wb/download/<token>` — both plain HTTP next to the socket, so a proxy needs no special rule beyond its body-size limit.

Not there yet: a `webbeans-maven-plugin` (`wb:run`, `wb:package`) and an archetype; automatic certificate renewal (ACME — use a proxy or an external tool that writes the key store, the server reads it at start); binary blob frames over the socket.

16. Porting from the NetBeans Platform

NetBeans	WebBeans
`JPanel`, `JButton`, …	`Panel`, `Button`, … (see Component Guide)
`Image`, `ImageUtilities`	`Resource` (`Resource.of("com/acme/icons/x.svg")`)
`javax.swing.Action`, `SystemAction`, `CallbackSystemAction`	`org.webbeans.actions.Action` (`actionPerformed(ActionEvent)` with `e.getLookup()`), registered actions, `@ActionRegistration(key = …)`
`java.awt.event.ActionListener` in `@ActionRegistration`	`org.webbeans.actions.ActionListener` (same constructor rules)
`Presenter.Menu/Toolbar/Popup`, `Actions.context/alwaysEnabled/callback/checkbox`	same names in `org.webbeans.actions`
`layer.xml`	annotations only (`@FolderRegistration` for menu/toolbar folders, `@LayoutRegistration` for layouts, `@HelpPage` for help)
`DialogDisplayer.notify` returning a value	`notify(...).thenAccept(...)`
`OptionsPanelController` + `@TopLevelRegistration`	same names in `org.webbeans.options`; `OptionsDisplayer.open()` returns a future
`HelpCtx`, `HelpCtx.Provider`, JavaHelp helpsets (`map.jhm`, `toc.xml`)	`org.webbeans.ui.HelpCtx` (same shape), `@HelpPage` + Markdown/HTML pages in the jar
`NotificationDisplayer`, `ProgressHandle`, `StatusDisplayer`	same names; session-wide, any thread
`SwingUtilities.invokeLater` / EDT	`ui.access(...)` / session executor
`Node.Cookie`, `OpenCookie`	`getLookup()` capabilities, `Openable`
`BeanTreeView`, `ChoiceView`	`TreeView`, `NodeComboBox`
`java.beans.PropertyEditor`	`org.webbeans.nodes.PropertyEditor` (kind + text conversion)
`FileObject` layer API, `Lookups.forPath`	`Registry`, `RegistryLookup.forPath`, `Actions.forPath`, `UserFiles`
`NbPreferences.forModule`	`WbPreferences.forModule` (per user)
`WindowManager` singleton, `Utilities.actionsGlobalContext()`	per browser tab (`WindowManager.getDefault()`), `Utilities.actionsGlobalContext()` = `wm.getGlobalContext()`
`TopComponent.getID()` / `WindowManager.findTopComponentID`	`getTopComponentId()` / `findTopComponentID`
`@NbBundle.Messages`, `NbBundle.getMessage`	`@org.webbeans.core.i18n.Messages` (same syntax), `Bundles.getMessage` — per-user locale
`--branding`, `Bundle_<brand>.properties`	the same
`JFreeChart` / `VisualVM` charts	`org.webbeans.charts.Chart` (server-side model, SVG renderer)

NetBeans	WebBeans
Java 3D / JOGL / JavaFX 3D	`org.webbeans.scene3d.Scene3D` (server-side scene graph, three.js renderer)
NetBeans' `NbModuleSuite` tests	`HeadlessUI`
`Lookup`, `@ServiceProvider`, `RequestProcessor`, `WeakListeners`	**unchanged** (NetBeans libraries)
Module manifests (`OpenIDE-Module`, dependencies, public packages), `ModuleInstall`	none — the class path is the application; `Installer` (via `@ServiceProvider`) for `restored()`/`closing()`/`close()`